

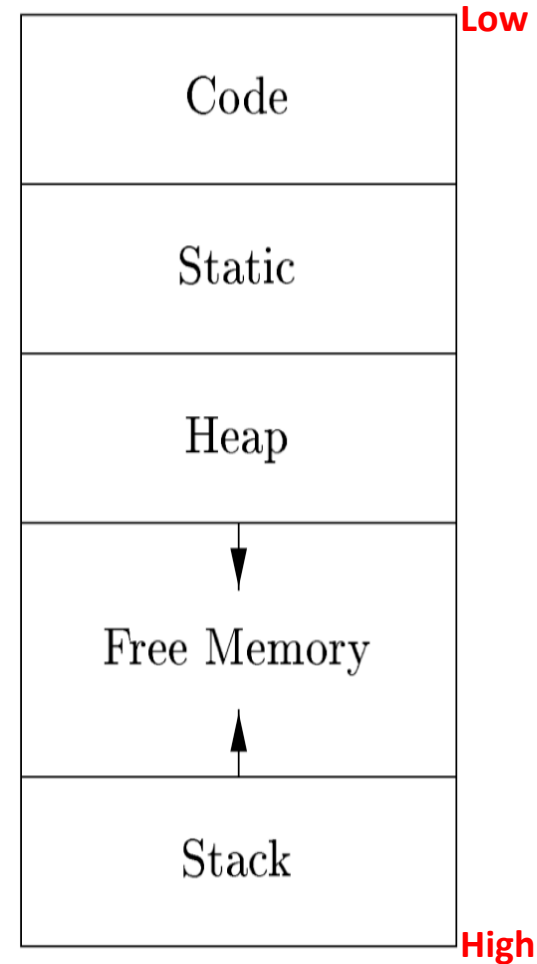
Garbage Collection

Chapter 13

Storage Organization

A typical way of subdividing memory is:

- **Code**: the executable target code
- **Static**: the data objects of which the size can be known at compile time, e.g.,
 - global constants
 - data generated by the compiler
- **Stack**: data structures called **activation records** that get generated during procedure calls
- **Heap**: data that is allocated and deallocated under program control
 - for C: *malloc*, *free*
 - for Java: *new*



Manual Memory Management

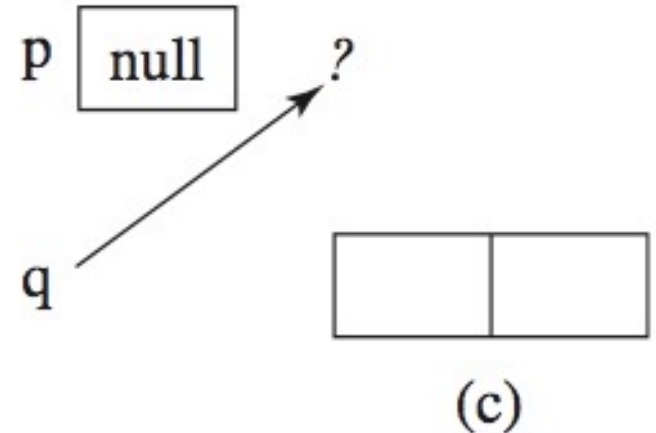
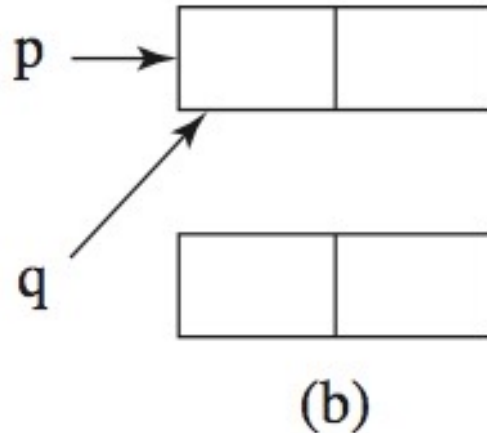
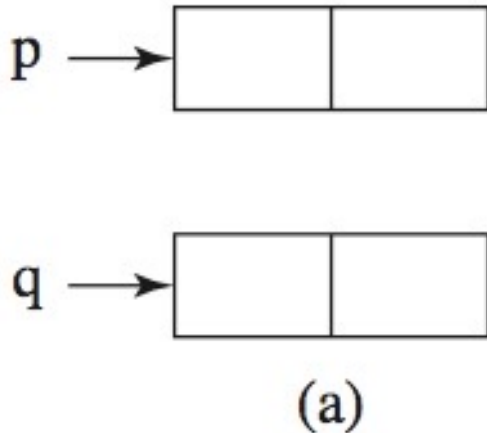
- Manual method
 - Approach used in C, C++
 - The use of `malloc` and `free` to perform dynamic allocation and de-allocation of pointer
- Problems with manual management
 - Easily lead to problems: memory leaks, double frees, use-after frees
 - Type-safe problems
- Storage bugs are hard to find
 - A bug can lead to a visible effect far away in time and program text from the source

Automatic Memory Management

- **Automatic Memory Management:** Reclamation of memory is automatic
- **Garbage:** Allocated but **no longer used** storage

```
class node {  
    int value;  
    node next;  
}  
node p, q;
```

```
p = new node();  
q = new node();  
q = p;  
delete p;
```



Garbage Collection: What

- **Garbage collection**: The process of reclamation of allocated but no longer used storage without an explicit call to free.
- Garbage collection is performed not by the compiler but by the runtime system (the support programs linked with the compiled code)

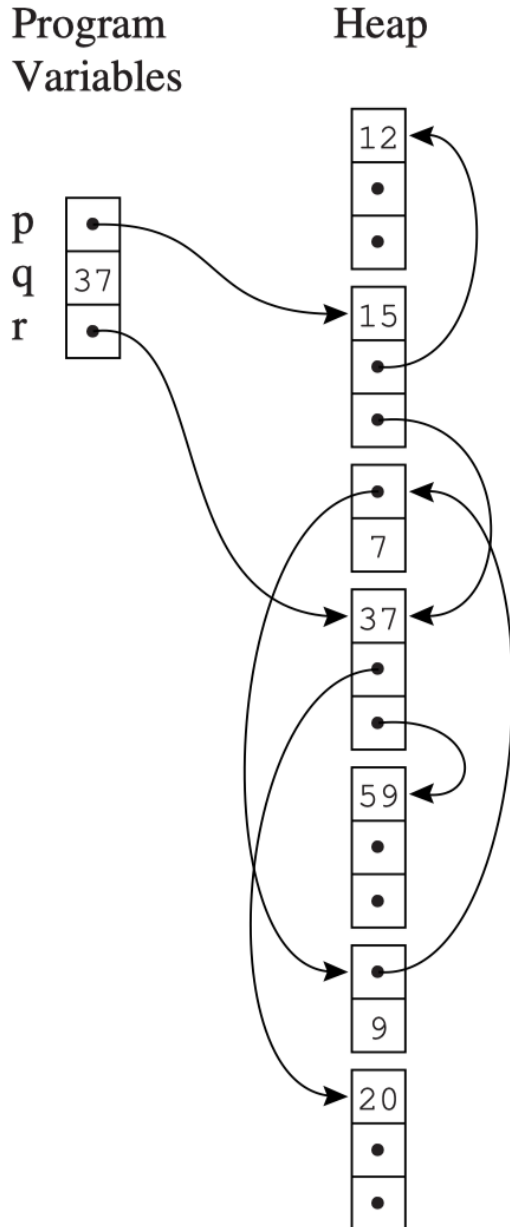
Garbage Collection: How

- **Garbage**: Allocated but **no longer used** storage
- Ideally, any record that is not dynamically live (will not be used in the future of the computation) is garbage.
- But it is **undecidable** whether an object is garbage
 - Need to rely on a conservative approximation
- **Idea**: Use reachability information as “approximation”
 - Heap-allocated records that are not **reachable** by any chain of pointers from program variables are **garbage**
 - Conservative: **not reachable** -> **garbage**
- **Garbage** -> **may be reachable**

Garbage Collection: How

- An object x is **reachable** if and only if
 - A register contains a pointer to x , or
 - another reachable object y contains a pointer to x

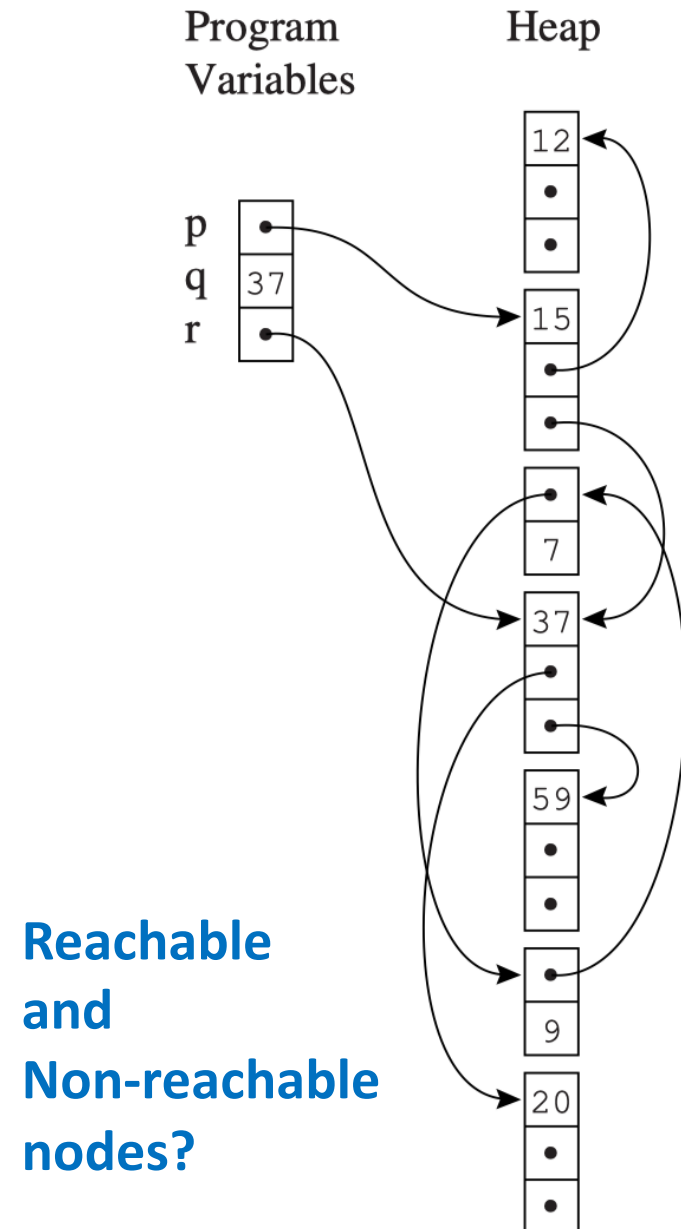
Basic Data Structure: Directed Graph



- Program variables and heap-allocated records form a **directed graph**.
 - **Nodes**
 - **Edges**: pointing relationship
- The program variables are the **roots** of this graph.
 - Registers
 - Local vars/formal parameters on stack
 - global variables
- A node **n** is **reachable** if there is a path of directed edges **r -> ... -> n**
 - **r**: some root

Garbage Collection: Example

```
let
  type list = {link: list, key: int}
  type tree = {key: int, left: tree,
right: tree}
  function maketree() = ...
  function showtree(t: tree) = ...
in
  let var x := list{link=nil, key=7}
    var y := list{link=x, key=9}
  in x.link := y
  end;
  let var p := maketree()
    var r := p.right
    var q := r.key
  in garbage-collect here
    showtree(r)
  end
end
```



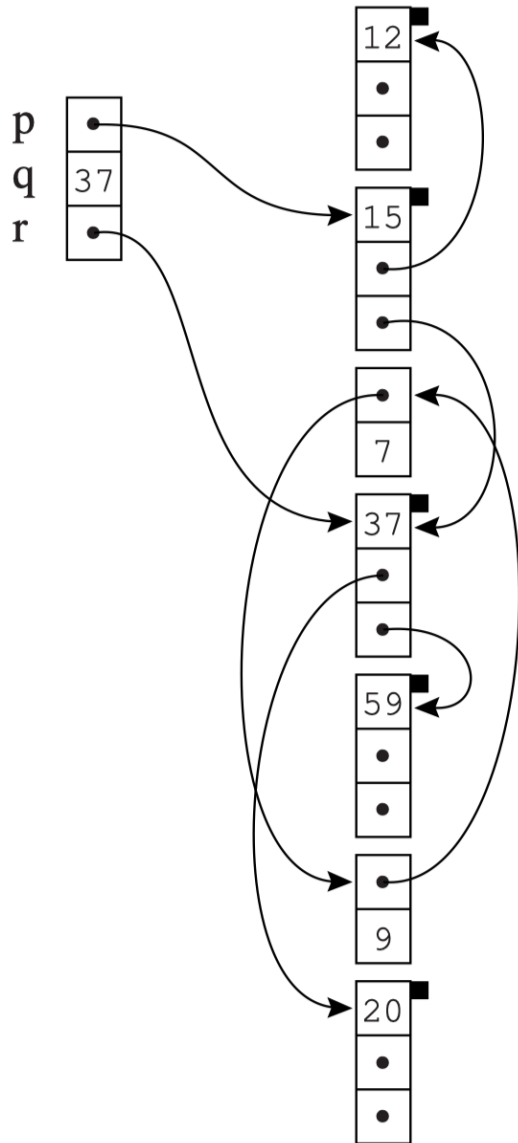
Outline

- Mark-and-Sweep Collection
- Reference Counts
- Copying Collection
- Interface to the Compiler

Outline

- **Mark-and-Sweep Collection**
- Reference Counts
- Copying Collection
- Interface to the Compiler

Mark-and-Sweep Collection

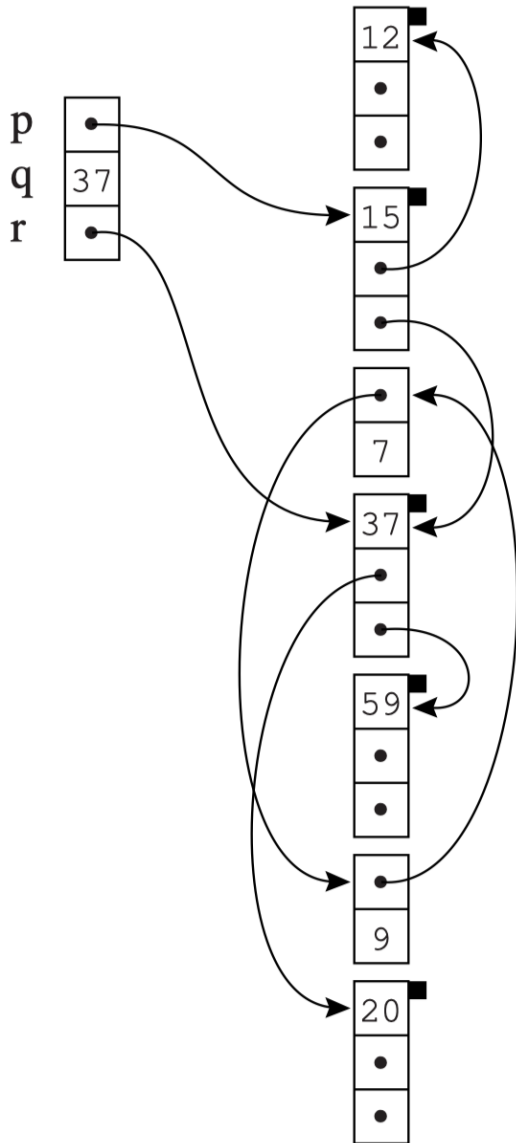


(a) Marked

- **Mark:**
 - Search the graph from the roots (program variables)
 - Mark the nodes that are visited
- A graph-search algorithm such as **depth-first search** can **mark** all the reachable nodes.

```
function DFS(x)
if x is a pointer into the heap
  if record x is not marked
    mark x
  for each field fi of record x
    DFS(x.fi)
```

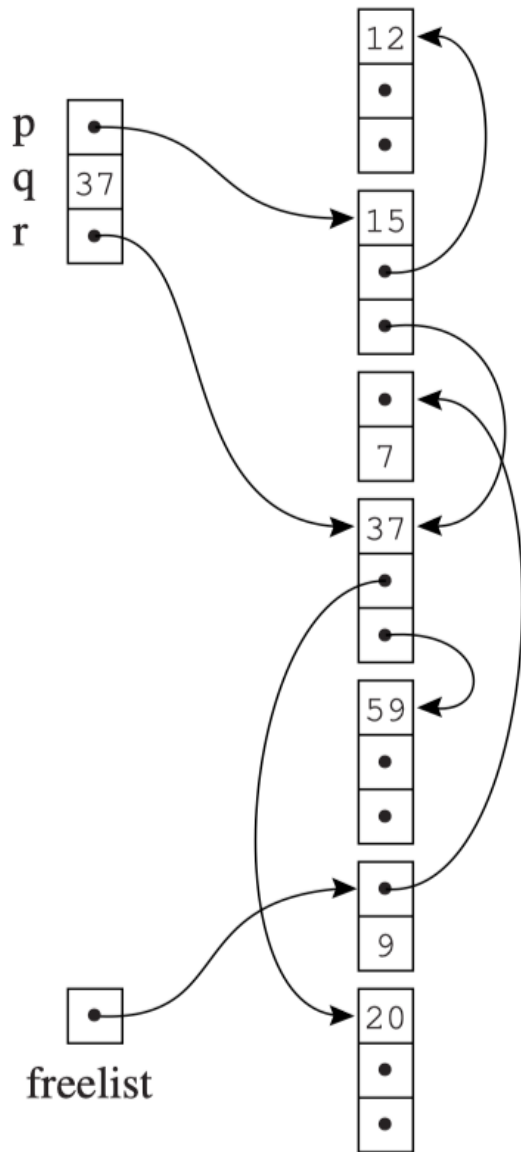
Mark-and-Sweep Collection



- Any node not marked must be **garbage**, and should be **reclaimed**.
- How?

(a) *Marked*

Mark-and-Sweep Collection



(b) Swept

- **Sweep:**

- Sweep the entire heap by a linear scan
- Link the unmarked nodes in a linked list (*freelist*)
- Unmark marked nodes for the next garbage collection

Mark-and-Sweep Collection - Algorithm

Mark phase:

for each root v
 DFS(v)

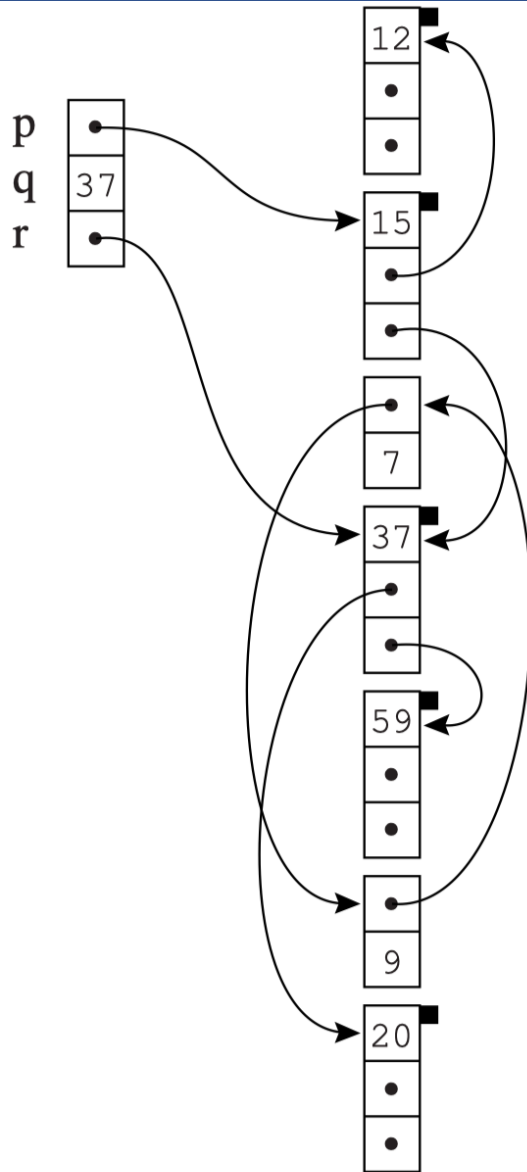
Sweep phase:

$p \leftarrow$ first address in heap
while $p <$ last address in heap
 if record p is marked
 unmark p
 else let f_1 be the first field in p
 $p.f_1 \leftarrow$ freelist
 freelist $\leftarrow p$
 $p \leftarrow p + (\text{size of record } p)$

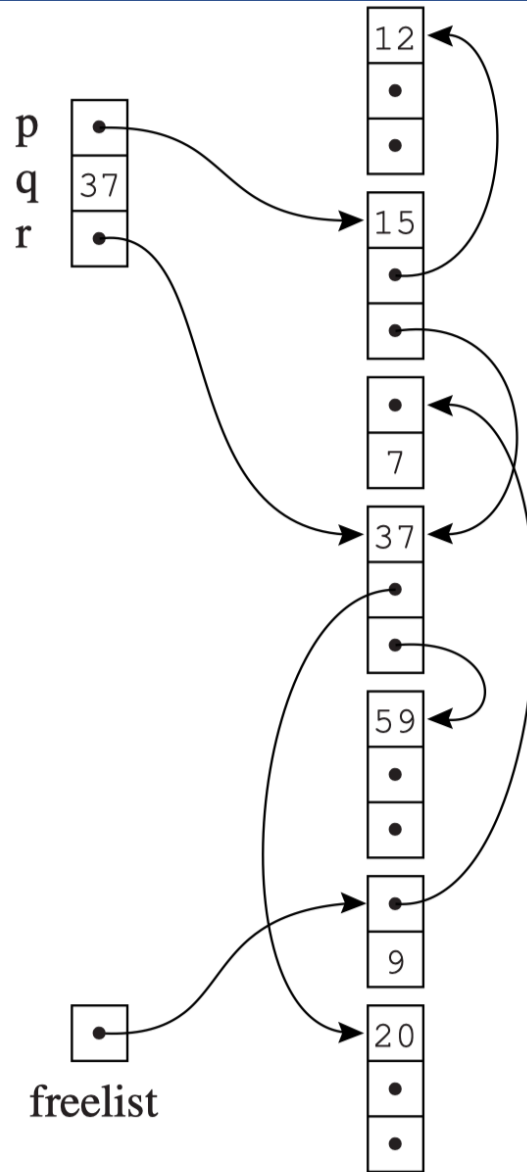
ALGORITHM 13.3. Mark-and-sweep garbage collection.

- After the garbage collection, the compiled program resumes execution.
- Whenever it wants to heap-allocate a new record, it gets a record from the *freelist*
- Do *garbage collection again* when *freelist* is empty.

Mark-and-Sweep Collection



(a) Marked



(b) Swept

- Mark
- Sweep
 - freelist
 - unmark

Cost of Mark-and-Sweep Collection

- H : heap size, R : the number of reachable nodes
- **Time of GC**
 - **Mark:** DFS takes time proportional to R
 - **Sweep:** takes time proportional to H .
 - Total time: $c_1R + c_2H$
- GC replenish the freelist with $H-R$ words
- Amortized cost $(c_1R + c_2H)/(H - R)$
 - If R is closed to H , the cost is very High

The Problem of DFS

- The **DFS** algorithm is **recursive**
 - Extreme case: **N** stack frames form an **N**-element linked list
 - The length of the stack of activation records for DHS would be larger than the entire heap.
- **Solution:** Use an **explicit stack** instead of recursion
 - **H** words instead of **H** activation records

Using an Explicit Stack

```
function DFS(x)
```

```
    if  $x$  is a pointer and record  $x$  is not marked
```

```
        mark  $x$ 
```

```
         $t \leftarrow 1$ 
```

```
        stack[t]  $\leftarrow x$  // push the start of DFS on stack
```

```
        while  $t > 0$ 
```

```
             $x \leftarrow \text{stack}[t]; t \leftarrow t - 1$  // pop an item from the stack
```

```
            for each field  $f_i$  of record  $x$ 
```

```
                if  $x.f_i$  is a pointer and record  $x.f_i$  is not marked
```

```
                    mark  $x.f_i$ 
```

```
                     $t \leftarrow t + 1; \text{stack}[t] \leftarrow x.f_i$ 
```

- t : the index of the stack top
- stack: a worklist

Using an Explicit Stack

```
function DFS(x)
```

```
  if  $x$  is a pointer and record  $x$  is not marked
```

```
    mark  $x$ 
```

```
     $t \leftarrow 1$ 
```

```
    stack[ $t$ ]  $\leftarrow x$  // push the start of DFS on stack
```

```
    while  $t > 0$ 
```

```
       $x \leftarrow \text{stack}[t]; t \leftarrow t - 1$  // pop an item from the stack
```

```
      for each field  $f_i$  of record  $x$ 
```

```
        if  $x.f_i$  is a pointer and record  $x.f_i$  is not marked
```

```
          mark  $x.f_i$ 
```

```
           $t \leftarrow t + 1; \text{stack}[t] \leftarrow x.f_i$ 
```

- Benefit: H words instead of H activation records
- However, it is still unacceptable to require auxiliary stack memory as large as the heap being collected.

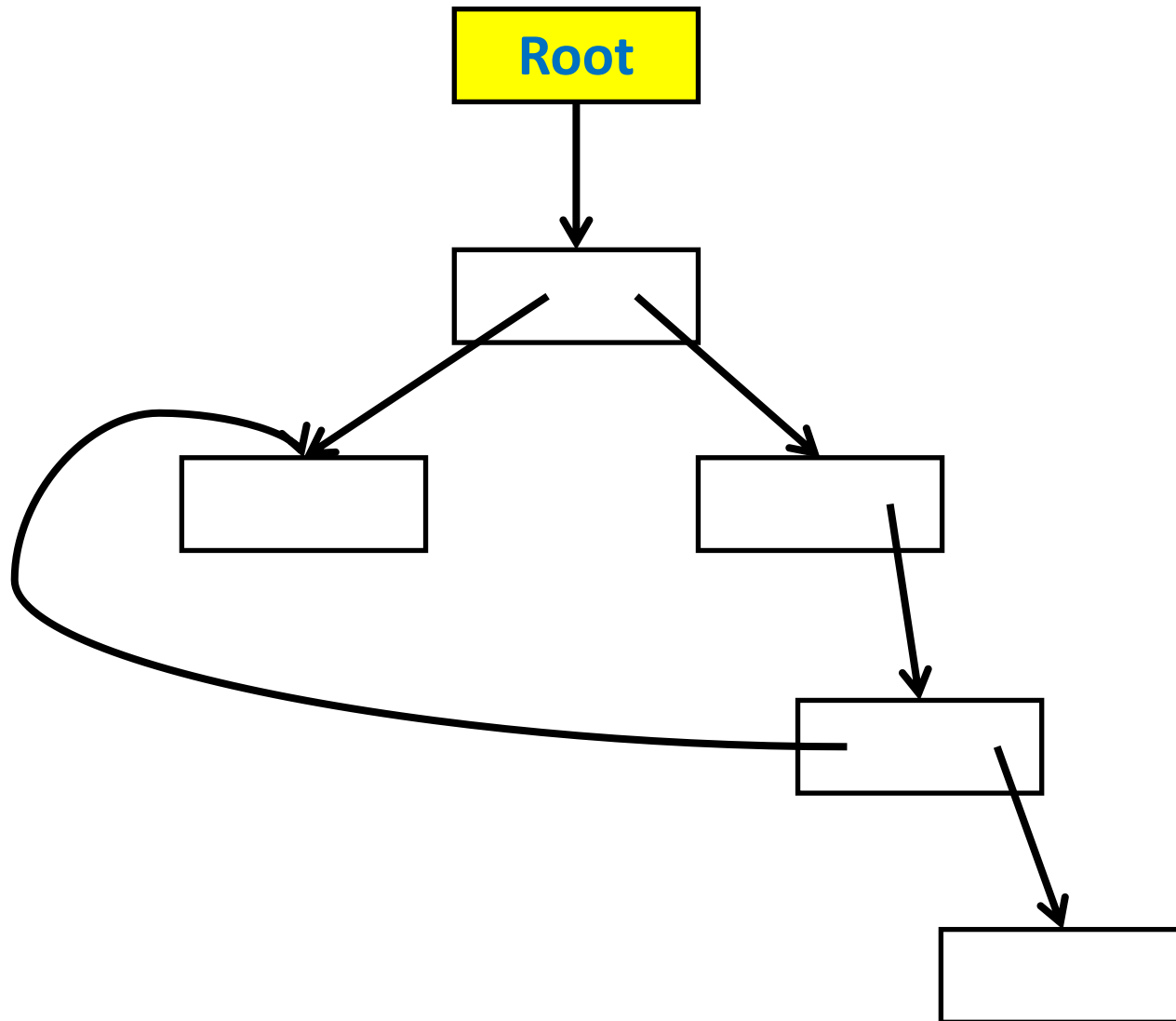
Pointer Reversal

- **Problem:** DFS needs a stack
 - The depth of the stack could be as large as the heap size
- **Solution:**
 - Don't use an explicit stack for DFS
 - Reuse the graph components to assist backtracking

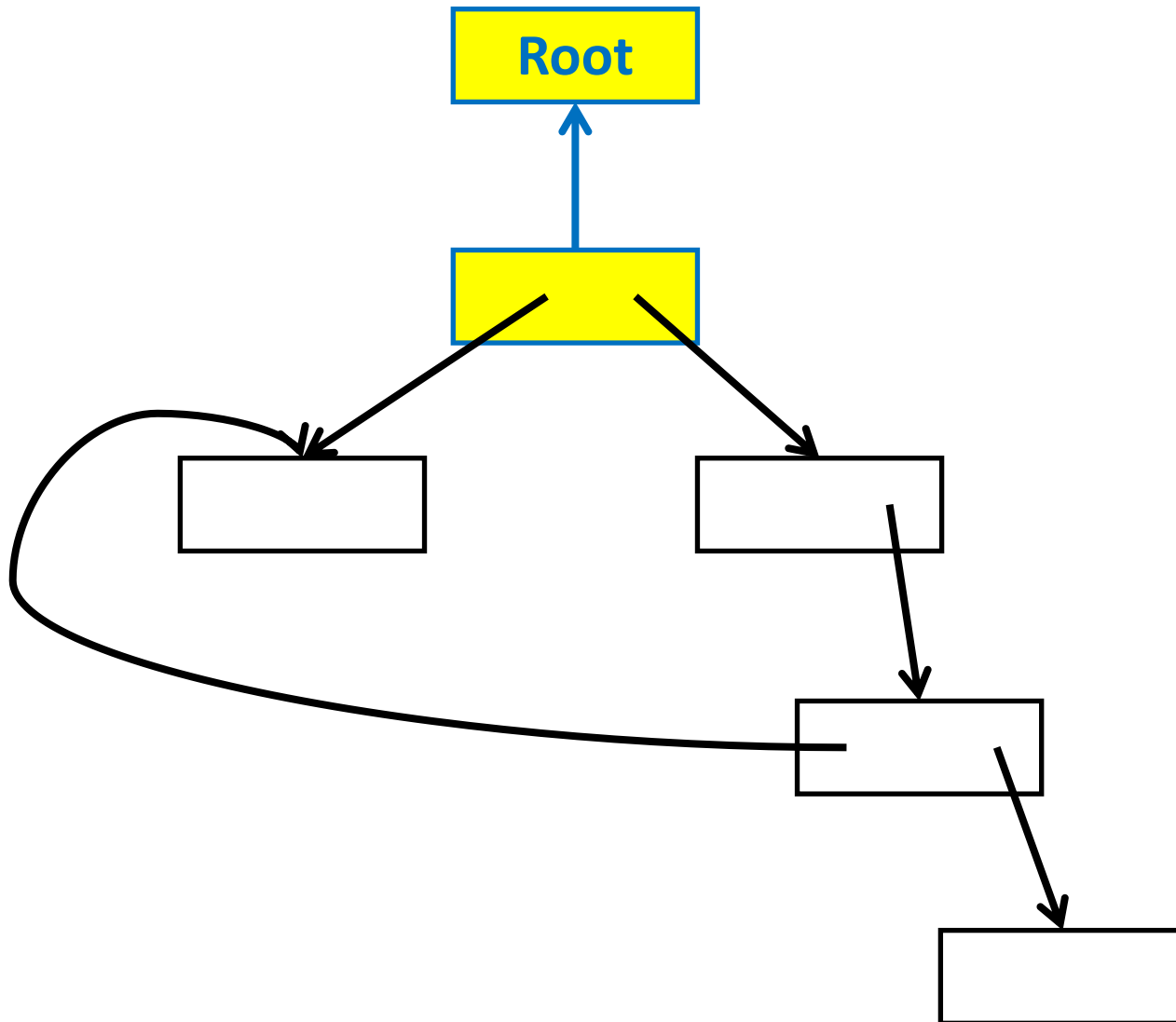
Pointer Reversal

- **The basic idea:** store the DFS stack in the directed graph itself
- When a new record is encountered during the search
 - Mark the record
 - Change a pointer in the record to point back to the DFS parent record (**pointer reversal**)
 - When we can go no deeper, return, following the back links, restoring the links

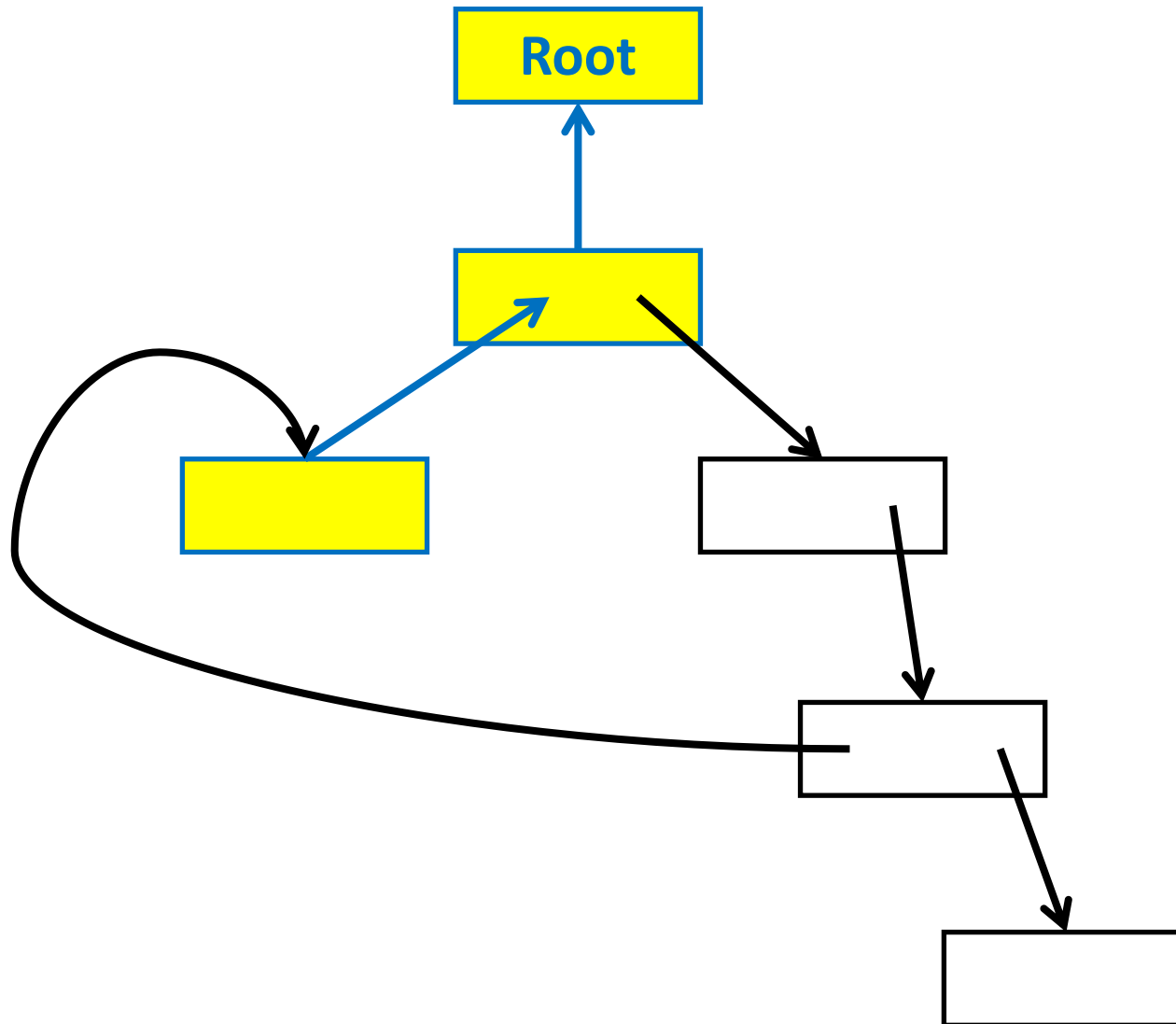
Pointer Reversal



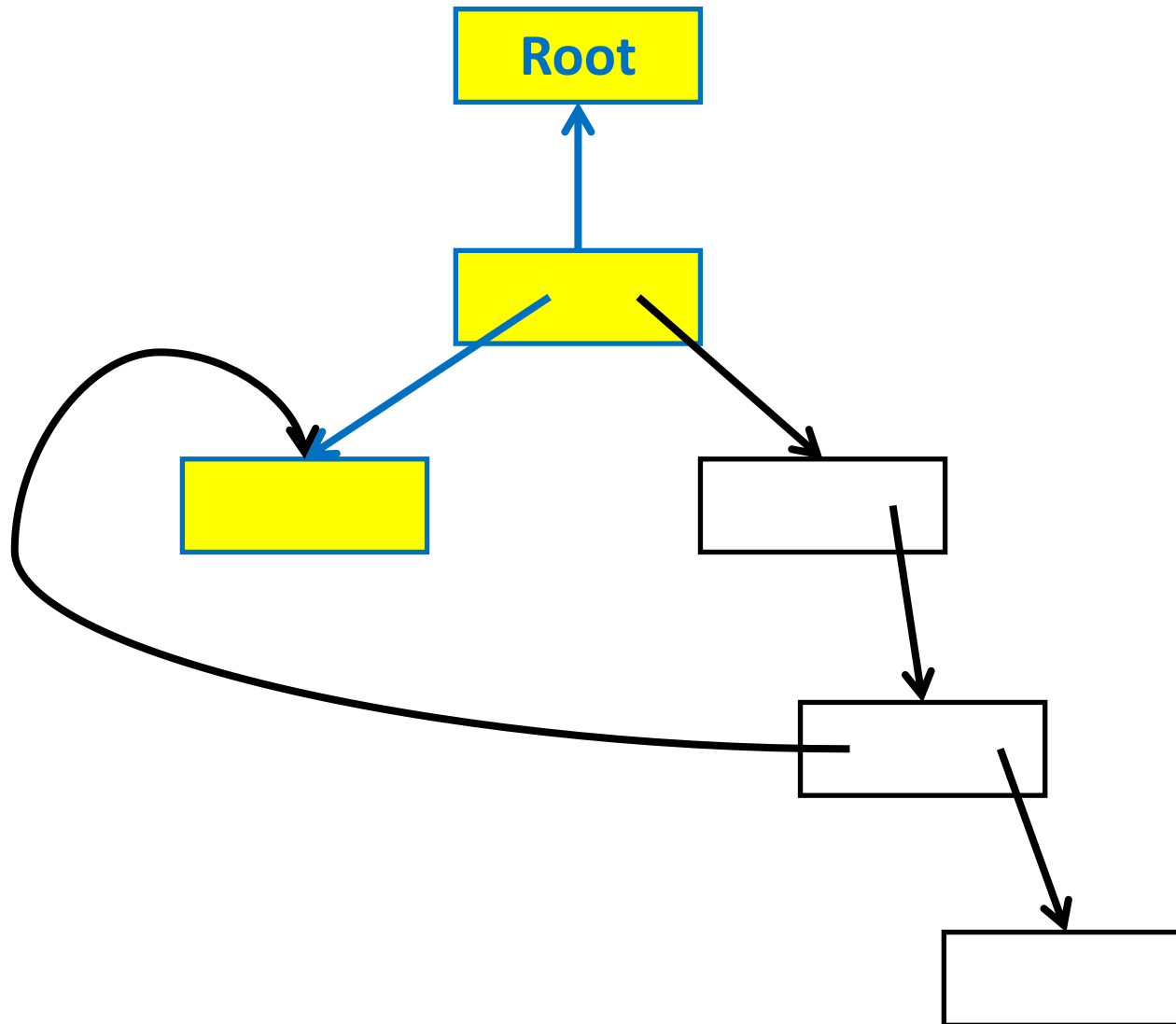
Pointer Reversal



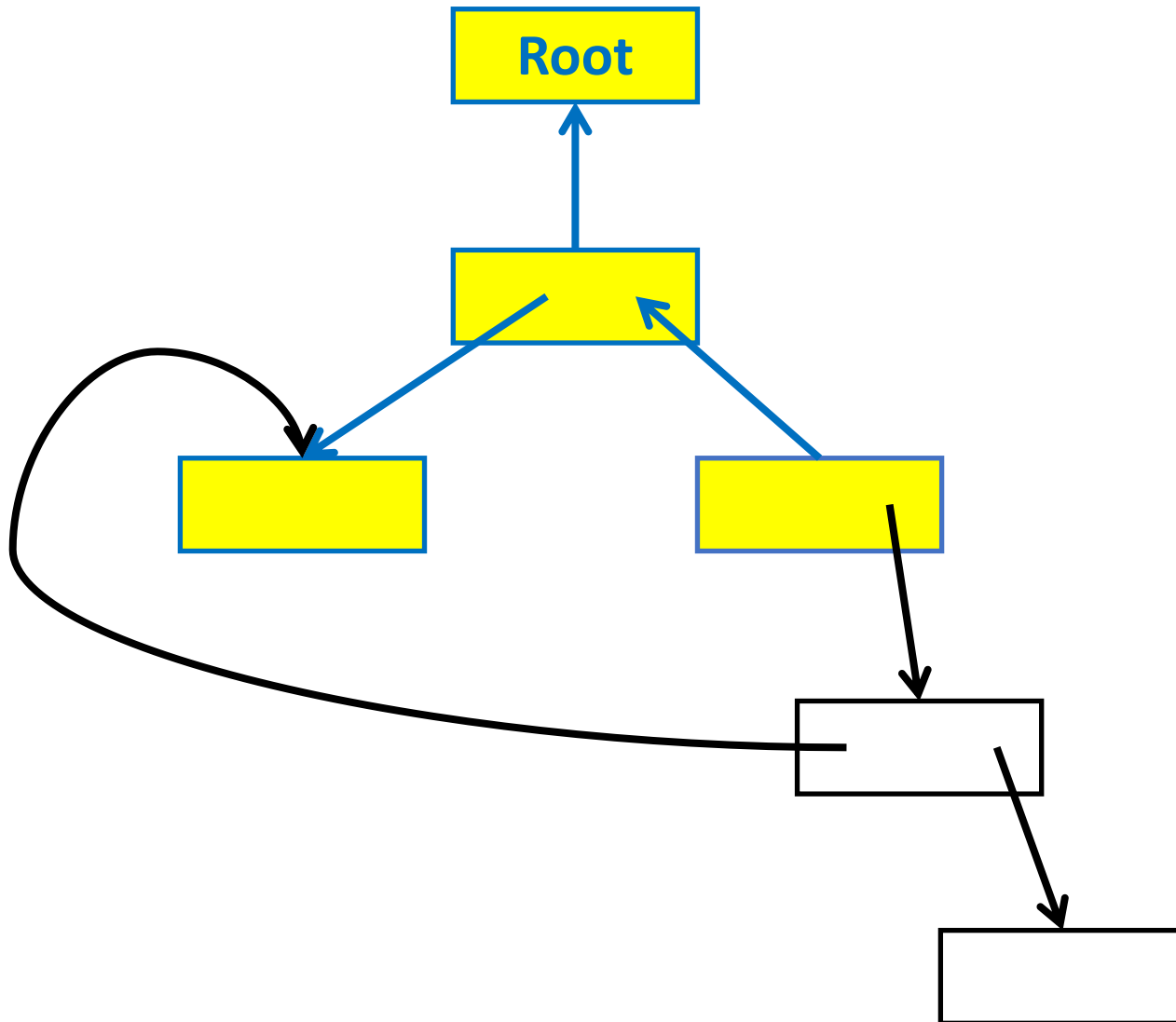
Pointer Reversal



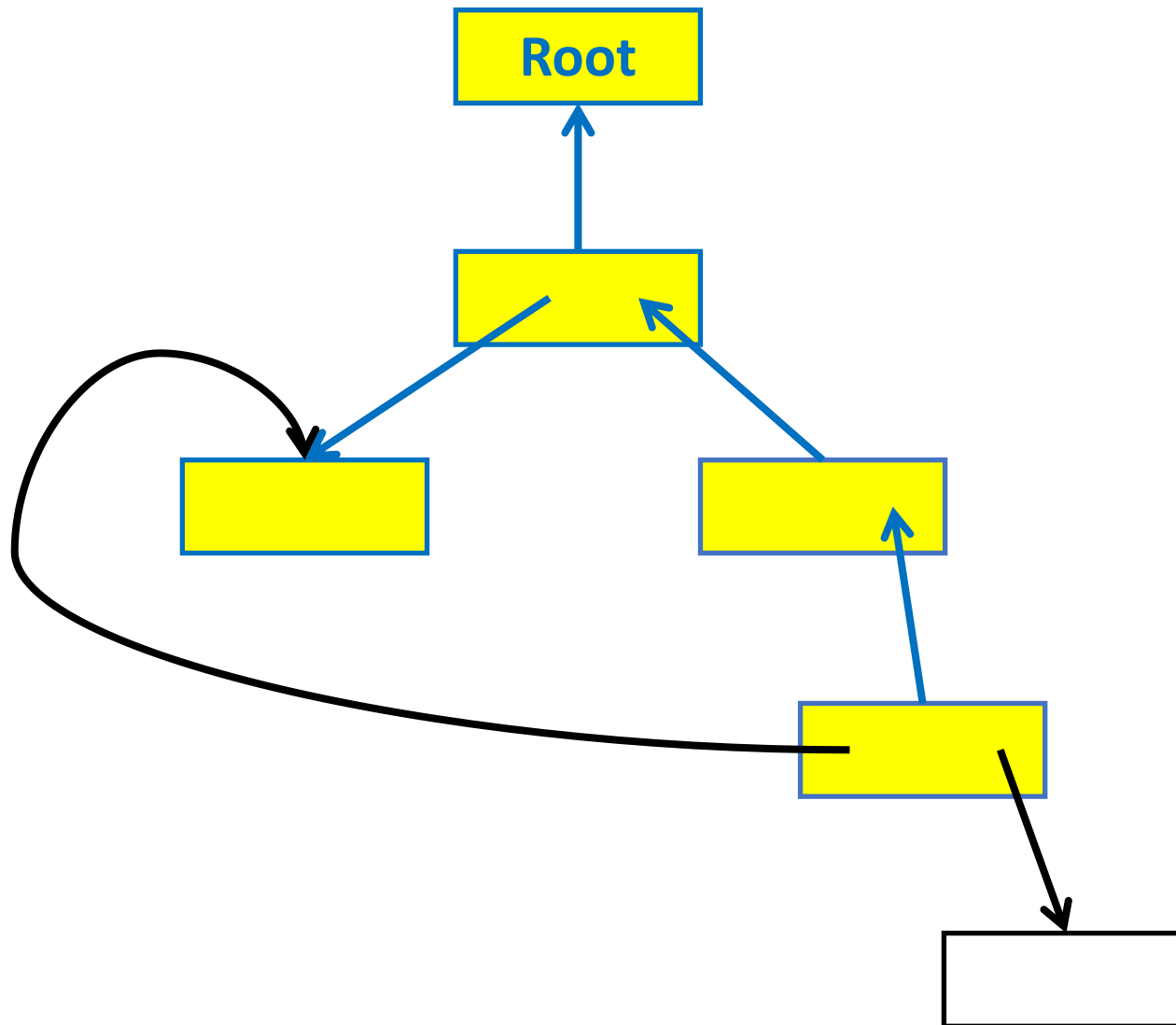
Pointer Reversal



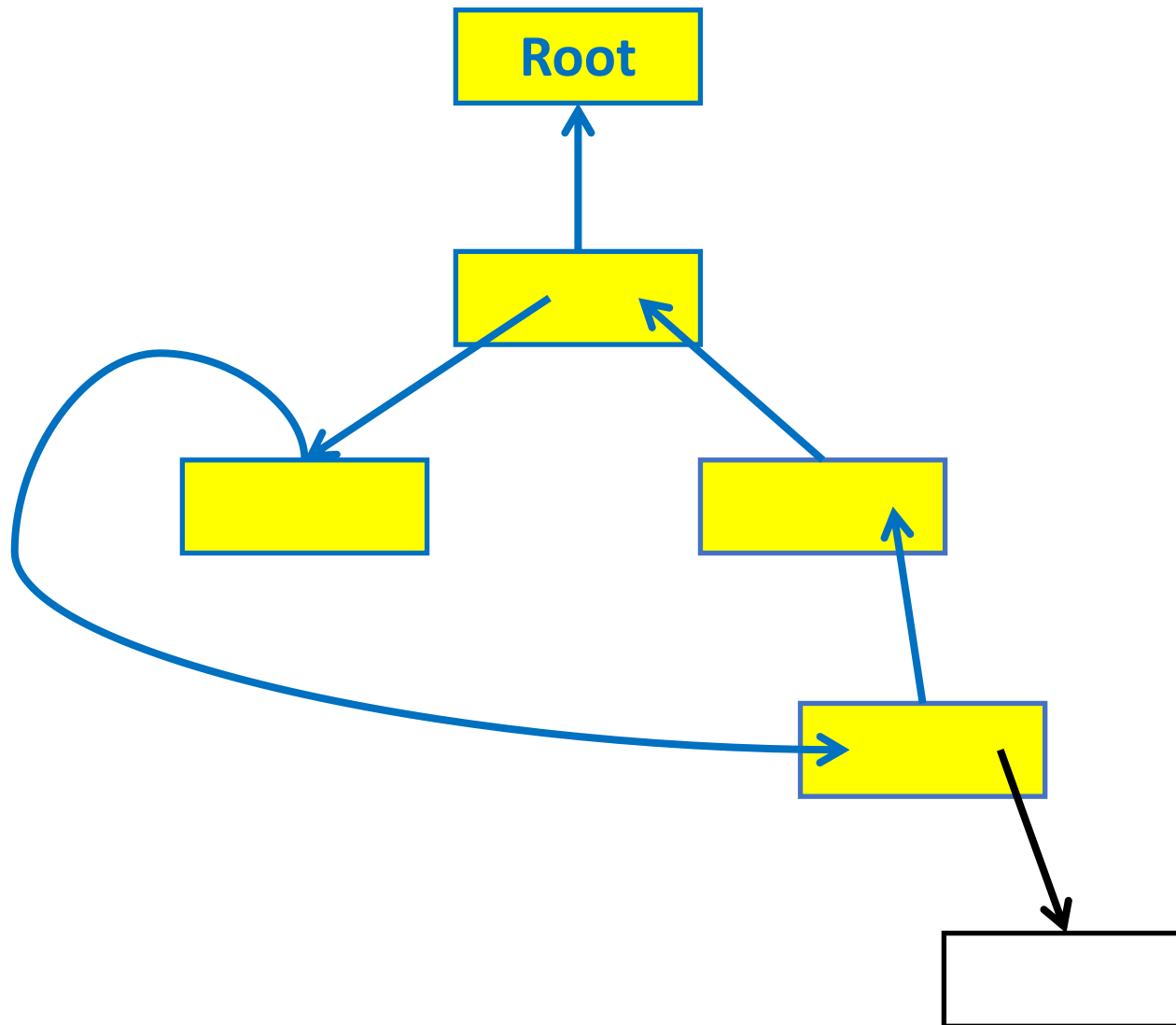
Pointer Reversal



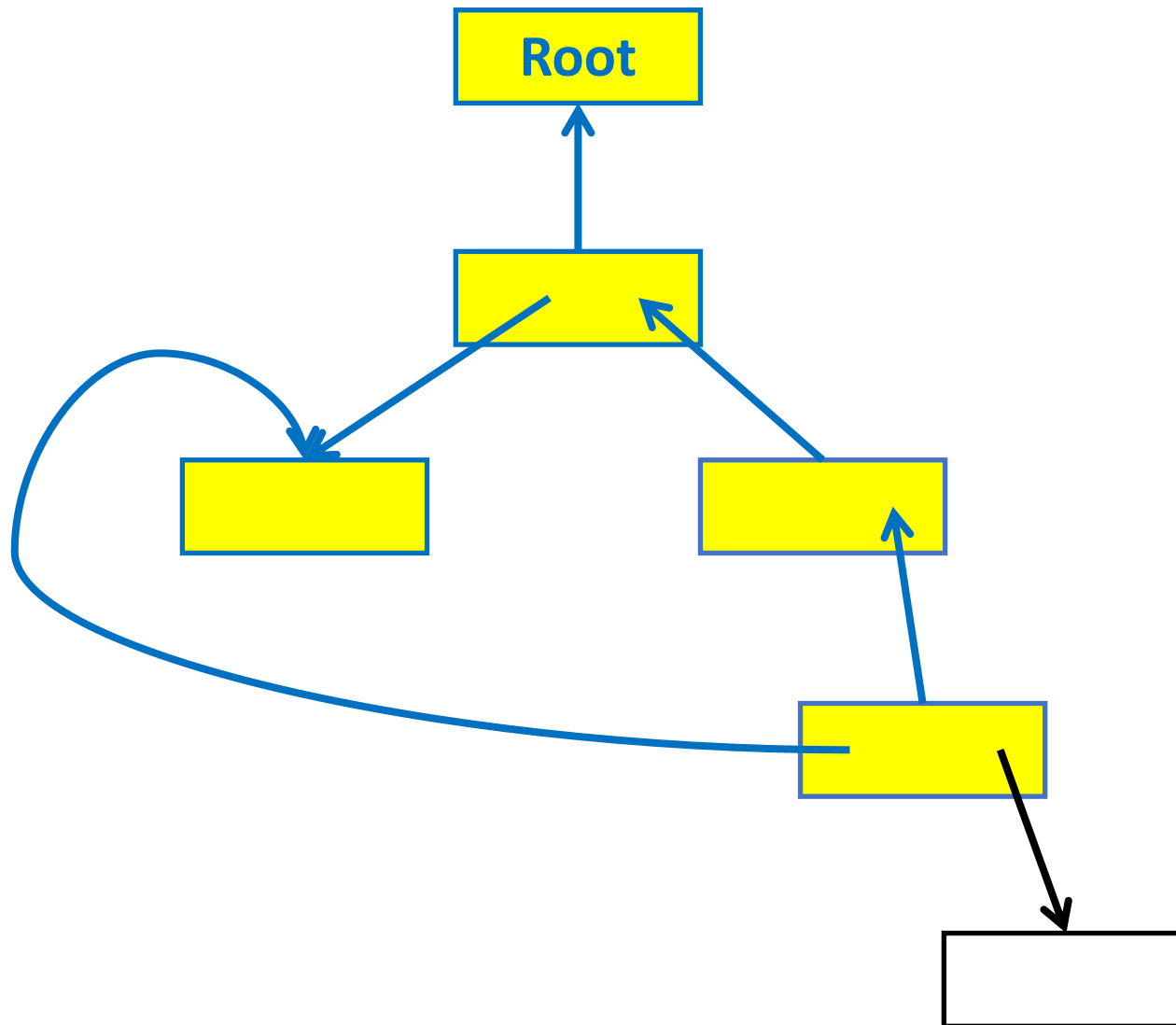
Pointer Reversal



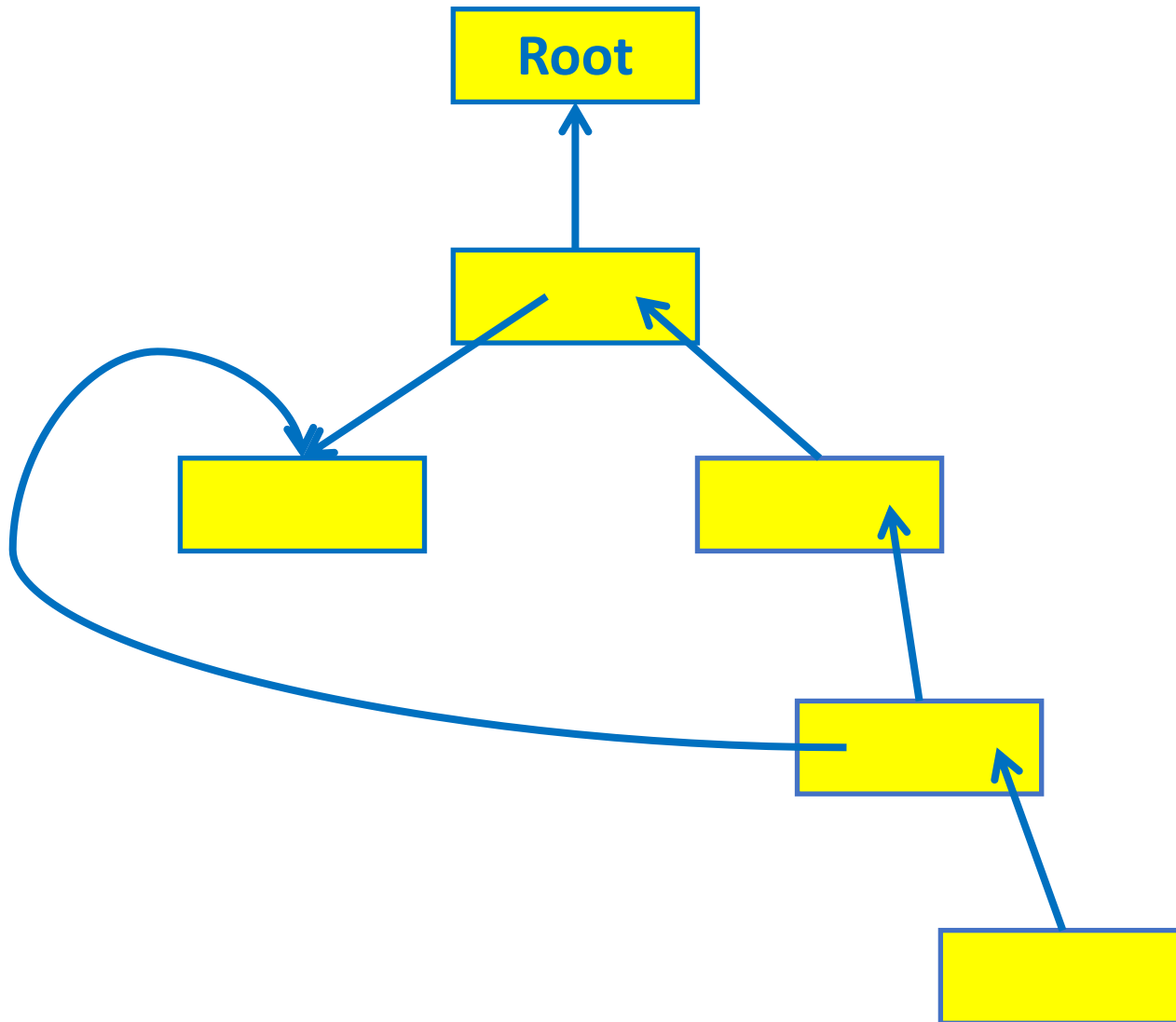
Pointer Reversal



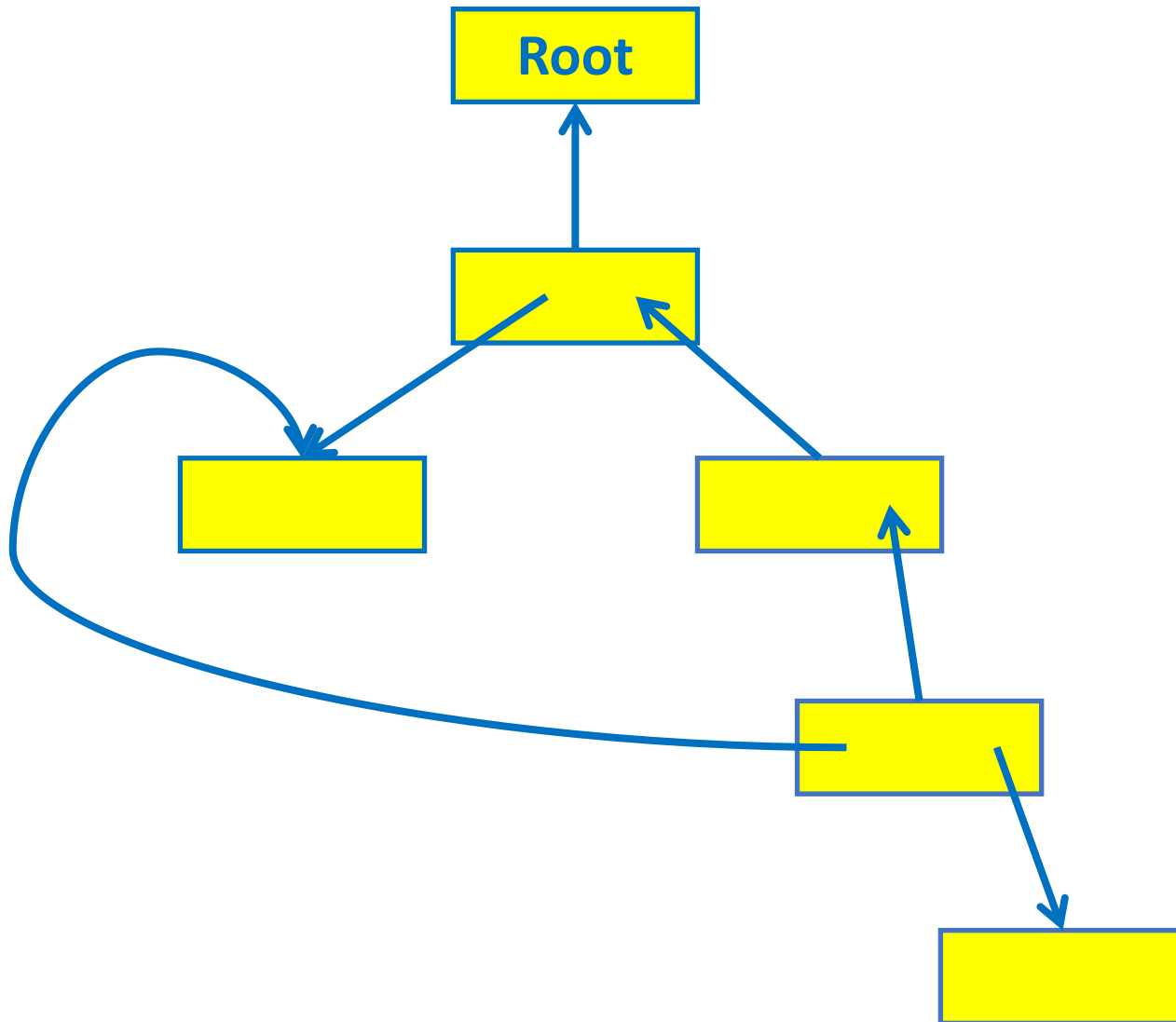
Pointer Reversal



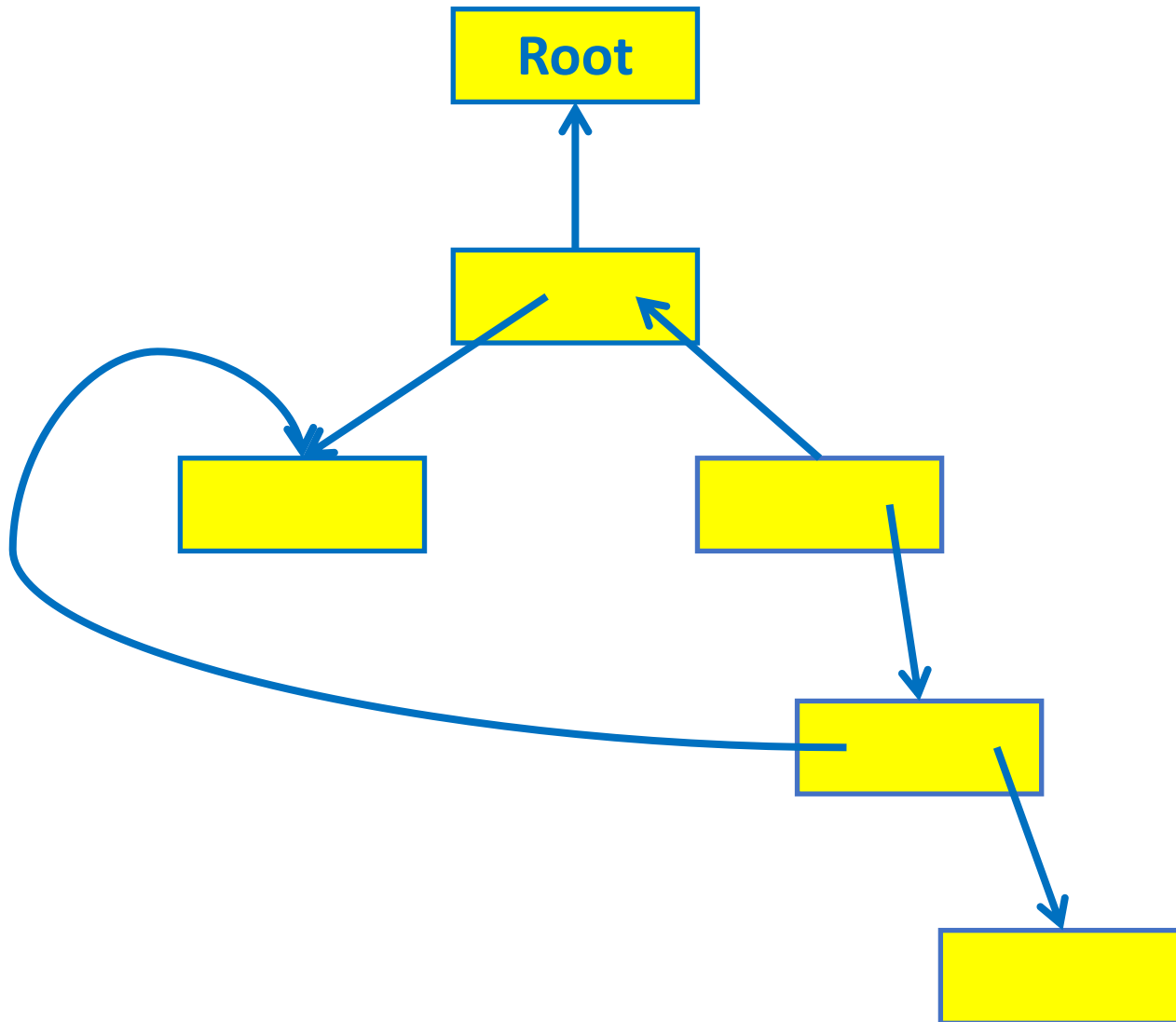
Pointer Reversal



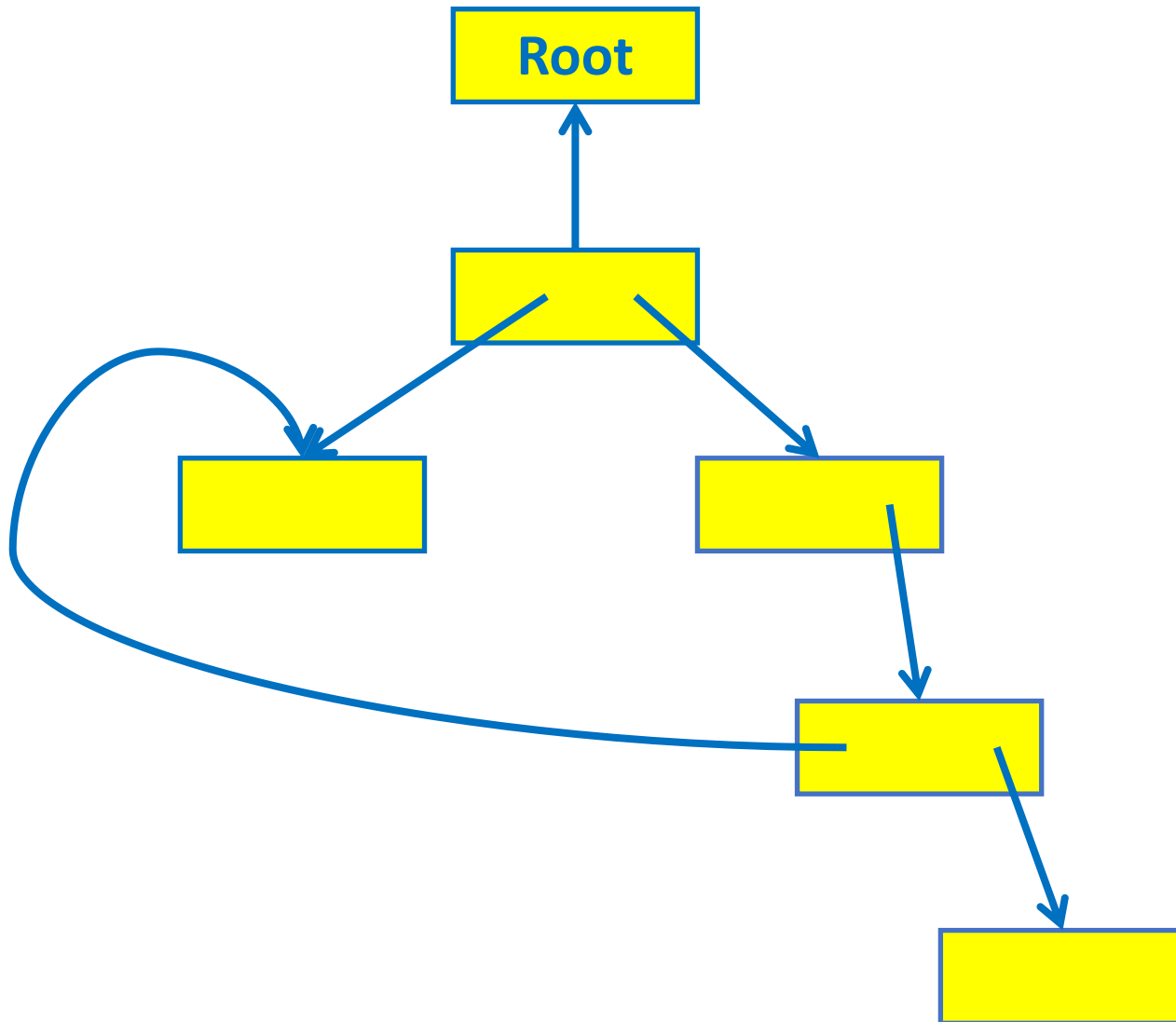
Pointer Reversal



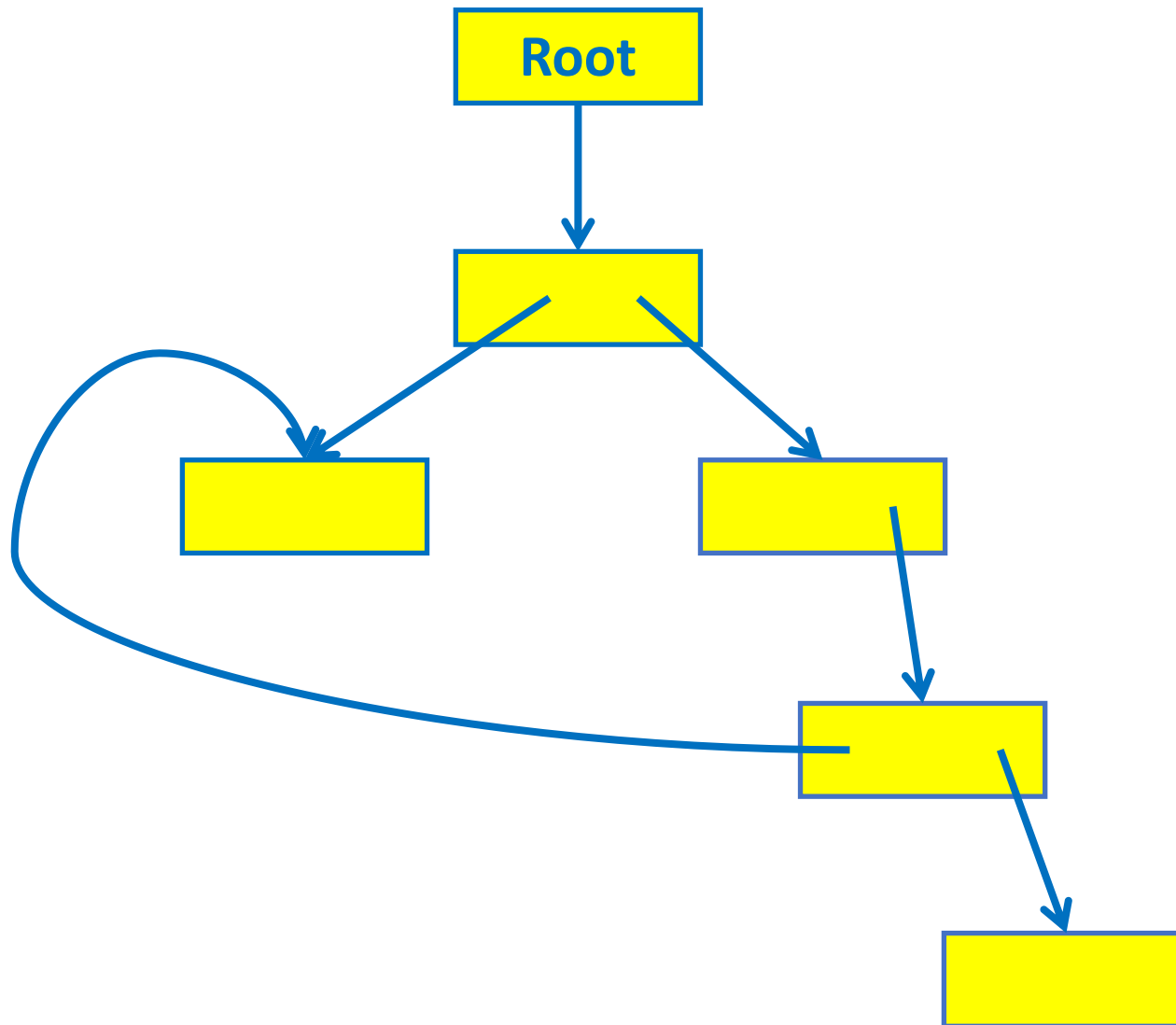
Pointer Reversal



Pointer Reversal



Pointer Reversal



Pointer Reversal

```
function DFS(x)  
if x is a pointer into the heap  
  if record x is not marked  
    mark x  
    for each field fi of record x  
      DFS(x.fi)
```

- Which pointer to reuse?
- Observation:
 - After the contents of field *x.fi* has been pushed on the stack, the algorithm will never again look the original location of *x.fi*
 - *x.fi* can be used to store one element of the stack
 - When the stack is popped, restore the original value of *x.fi*

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

/ initialization */*

while true

...

if $i < \#$ of fields in record x

/ process the i _th field */*

else

/ decide termination*

& back-track to the DFS parent record of x ./*

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.fi$

if y is a pointer and record y is not marked

$x.fi \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.fi$; $x.fi \leftarrow y$

$\text{done}[x] \leftarrow i + 1$

- **done**: how many fields in each record have been processed
- **t**: the top of the stack

To better understand, suppose y points to record1

// let $x.fi$ point to the old top of stack (its DFS parent) ; Push x to the stack; Start to process the stored $x.fi$

// store x to y , pop the top of the stack to x

// back to the root node, terminate

// restore the correct link

// start to process the next field of the DFS parent

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.fi$

if y is a pointer and record y is not marked

$x.fi \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

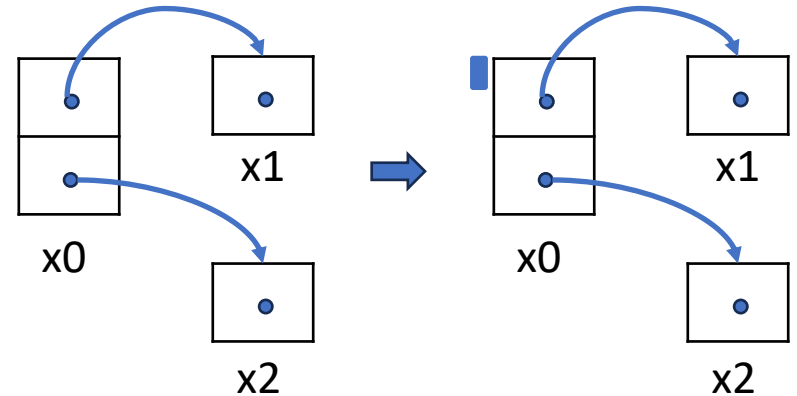
$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.fi$; $x.fi \leftarrow y$

$\text{done}[x] \leftarrow i + 1$



$t = \text{nil}$

$x = x_0$

mark x_0

$\text{done}[x_0] = 0$

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.fi$

if y is a pointer and record y is not marked

$x.fi \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

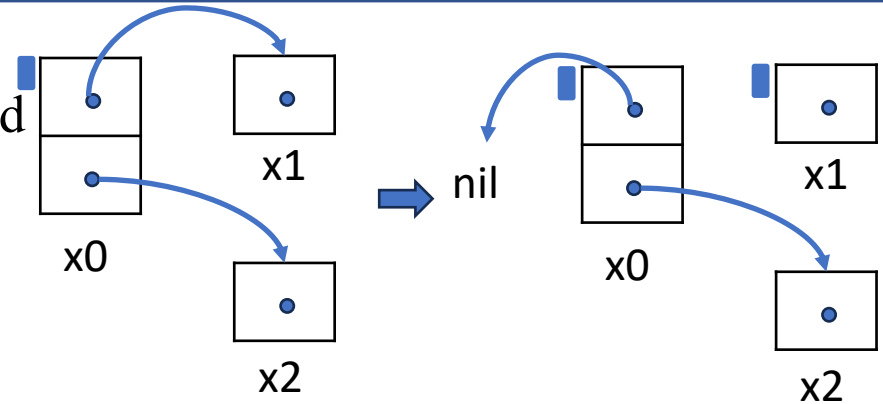
$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.fi$; $x.fi \leftarrow y$

$\text{done}[x] \leftarrow i + 1$



$t = \text{nil}$

$x = x0$

$i = \text{done}[x0] = 0$

$y = x0.fi = x1$

$x0.fi = t = \text{nil}$

$t = x = x0$

$x = y = x1$

mark $x1$

$\text{done}[x1] = 0$

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.fi$

if y is a pointer and record y is not marked

$x.fi \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

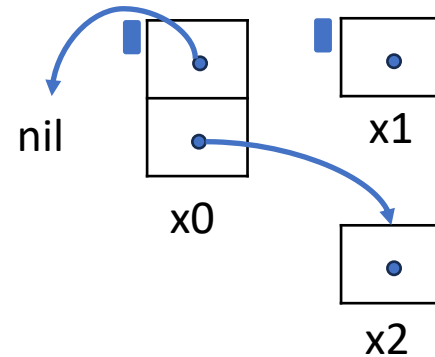
$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.fi$; $x.fi \leftarrow y$

$\text{done}[x] \leftarrow i + 1$



$t = x0$

$x = x1$

$i = \text{done}[x1] = 0$

$y = x1.fi$, not pointer

$\text{done}[x1] = i + 1 = 1$

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.f_i$

if y is a pointer and record y is not marked

$x.f_i \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

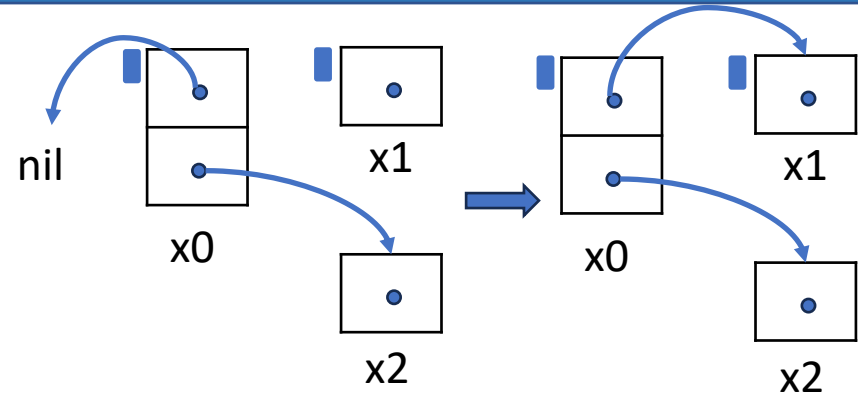
$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.f_i$; $x.f_i \leftarrow y$

$\text{done}[x] \leftarrow i + 1$



$t = x0$

$x = x1$

$i = \text{done}[x1] = 1$

$y = x1, x = x0$ // back to parent

$i = \text{done}[x0] = 0$

$t = x0.f_0 = \text{nil}$ // update stack top

$x0.f_0 = x1$ // restore

$\text{done}[x0] = 1$

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.f_i$

if y is a pointer and record y is not marked

$x.f_i \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

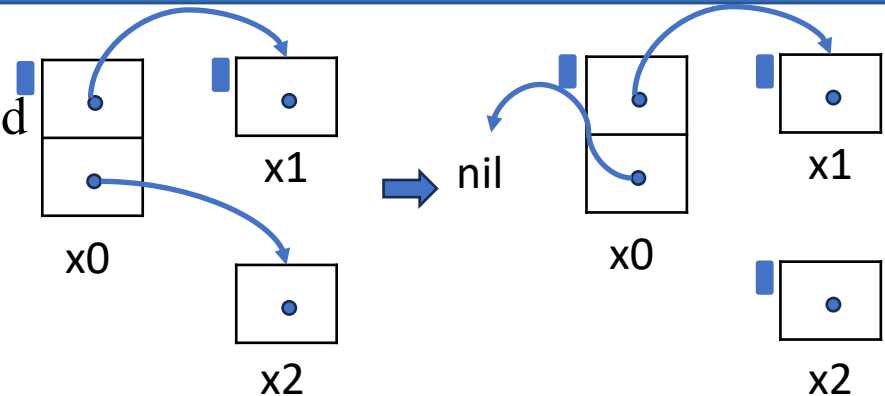
$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.f_i$; $x.f_i \leftarrow y$

$\text{done}[x] \leftarrow i + 1$



$t = \text{nil}$

$x = x0$

$i = \text{done}[x0] = 1$

$y = x0.f_1 = x2$

$x0.f_1 = t = \text{nil}$

$t = x = x0$

$x = y = x2$

mark $x2$

$\text{done}[x2] = 0$

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.fi$

if y is a pointer and record y is not marked

$x.fi \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

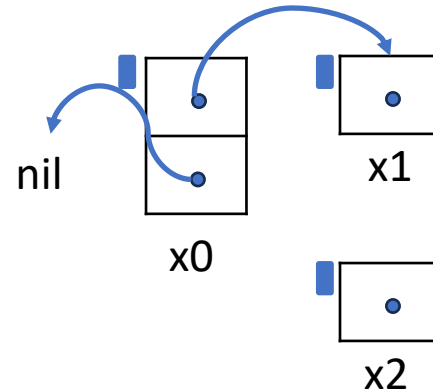
$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.fi$; $x.fi \leftarrow y$

$\text{done}[x] \leftarrow i + 1$



$t = x_0$

$x = x_2$

$i = \text{done}[x_2] = 0$

$y = x_2.fi$, not a pointer

$\text{done}[x_2] = 1$

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.f_i$

if y is a pointer and record y is not marked

$x.f_i \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

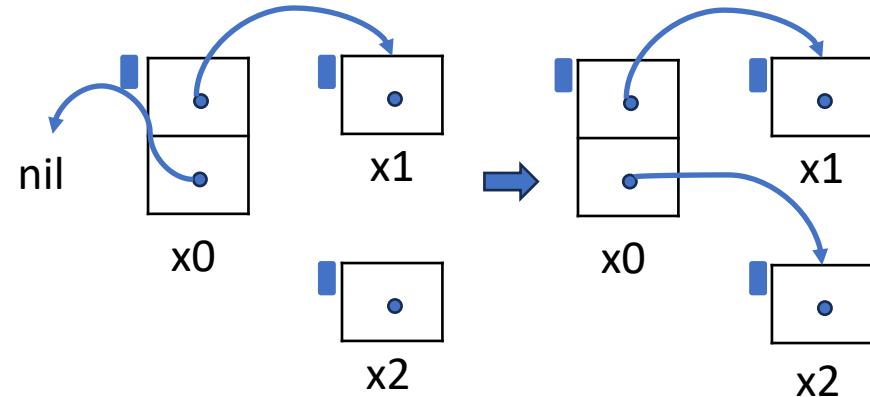
$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.f_i$; $x.f_i \leftarrow y$

$\text{done}[x] \leftarrow i + 1$



$t = x0$

$x = x2$

$i = \text{done}[x2] = 1$

$y = x = x2$

$x = t = x0$

$i = \text{done}[x0] = 1$

$t = x0.f1 = \text{nil}$

$x0.f1 = y = x2$ // restore

$\text{done}[x0] = 2$

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.fi$

if y is a pointer and record y is not marked

$x.fi \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

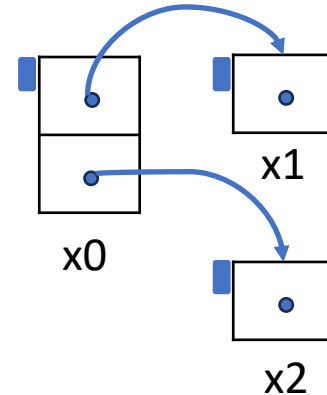
$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.fi$; $x.fi \leftarrow y$

$\text{done}[x] \leftarrow i + 1$



$t = \text{nil}$

$x = x0$

$i = \text{done}[x0] = 2$

$y = x = x0$

$x = t = \text{nil}$

return

Summary: Mark-and-Sweep

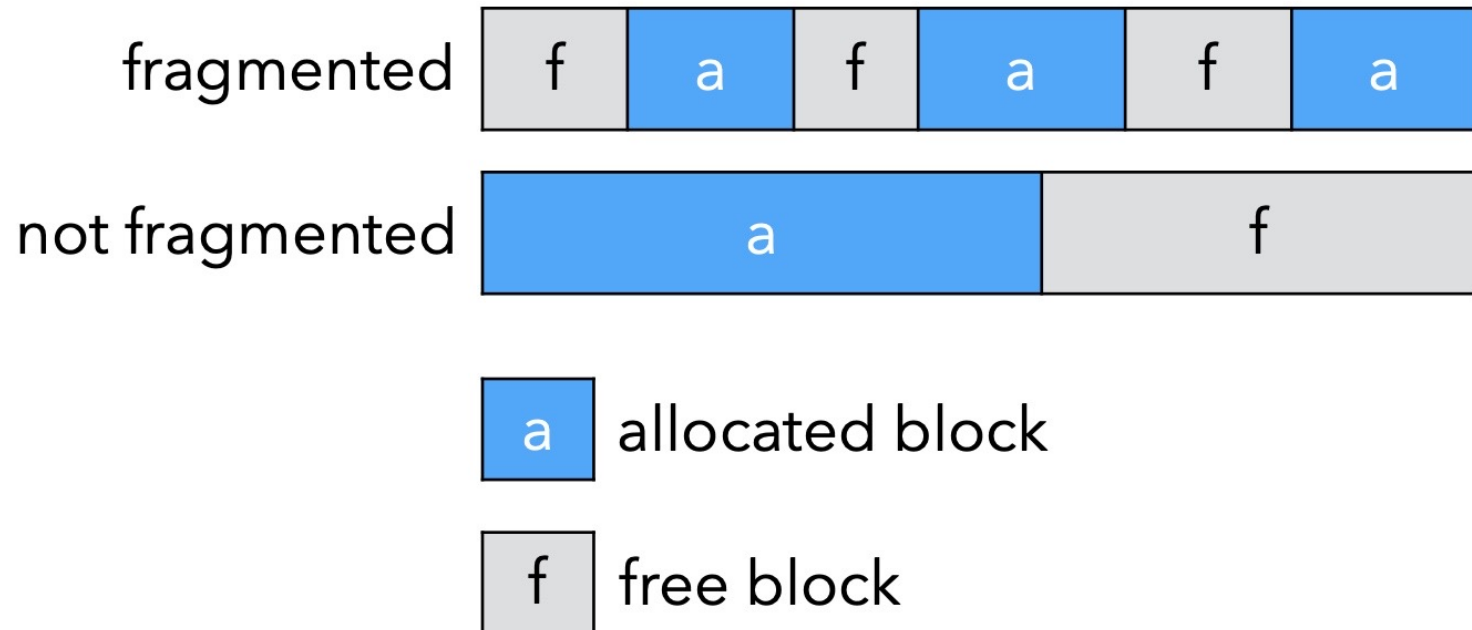
- Pros
 - High efficiency if little garbage exist
 - Be able to collect cyclic references
 - Objects/records are not moved during GC
- Cons
 - Low efficiency with large amount of garbage
 - Normal execution must be suspended
 - Leads to **fragmentation** in the heap
 - Cache misses, more complex allocation

About Fragmentation

- External fragmentation:
 - The program wants to allocate a record of size n , and there are many free records smaller than n but none of the right size
- Internal fragmentation:
 - The program uses a too-large record without splitting it, so that the unused memory is inside the record instead of outside.

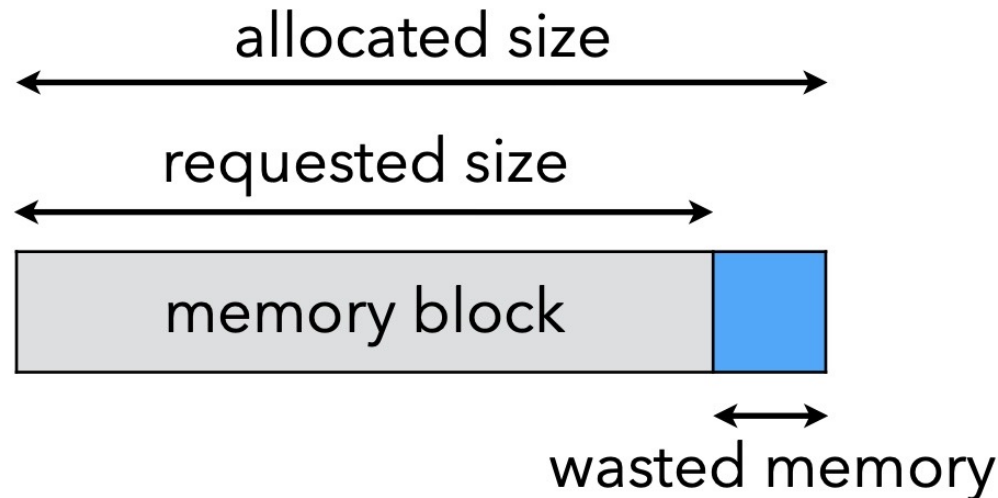
External Fragmentation: Example

- The two heaps below have the same amount of free memory, but the first suffers from external fragmentation, while the second does not.
- Therefore, some requests can be fulfilled by the second but not the first.



Internal Fragmentation: Example

- The memory manager sometimes allocates more memory than requested
 - E.g., to satisfy alignment constraints.
- This results in small amounts of wasted memory scattered in the heap, and is called internal fragmentation.



Homework

- **13.2**